

Complementi di Programmazione

Esercizi 2

CdL Informatica - Università degli studi di Modena e Reggio Emilia
AA 2024/2025

Angelo Ferrando

Esercizio 1

Scrivere un programma che chieda all'utente di immettere una serie di valori terminata dal simbolo '#'

Dopodichè applica una funzione a vostro piacere su tutti gli elementi.

Non usare un ciclo per l'applicazione della funzione

Esercizio 2

Modificare il programma precedente in modo che l'utente possa inserire solo numeri.

Gestire gli altri casi tramite eccezioni.

Esercizio 3

Modificare il programma precedente utilizzando una lambda function.

Esercizio 4

Creare una lista di numeri e utilizzare una funzione anonima (**lambda**) insieme a **filter** per ottenere una nuova lista contenente solo i numeri pari.

Stampare la lista risultante.

Successivamente, utilizzare la funzione **map** per ottenere una lista di quadrati dei numeri pari filtrati.

Esercizio 5

Creare una funzione chiamata **concatenate_strings** che

1. accetta un numero variabile di stringhe come argomenti posizionali (***args**)
2. e una stringa separatore opzionale con keyword argument (separator).

La funzione deve concatenare tutte le stringhe passate utilizzando il separatore indicato.

Esempio:

```
$- concatenate_strings('Hello', 'World', separator='-')
```

```
$- "Hello-World"
```

hint: Usare **reduce** (<https://docs.python.org/3.0/library/functools.html>)

Esercizio 6

Scrivere una funzione chiamata **create_multiplier** che

1. accetta un numero come argomento e
2. restituisce una closure.

Questa **closure** deve accettare un altro numero e restituire il prodotto dei due numeri.

Esempio:

```
$- multiply_by_3 = create_multiplier(3)  
$- print(multiply_by_3(10)) # Output: 30
```

Esercizio 7

Utilizzare un'espressione **generatrice** per creare un **generatore** che produca tutti i numeri **dispari** tra 0 e un numero n fornito dall'utente.

Utilizzare il generatore per stampare uno a uno tutti i numeri dispari generati.

Esercizio 8

Implementare un decoratore chiamato **@log_execution** che

1. ogni volta che la funzione decorata viene eseguita, scrive su un file di log (es. "execution_log.txt") la data e l'ora di esecuzione, insieme al nome della funzione eseguita e agli argomenti passati.
2. Applicare questo decoratore a una funzione che accetta un qualsiasi numero di argomenti e li somma.

Esercizio 9

Scrivere un decoratore che arricchisca una funzione come segue:

- 1) Oltre a ritornare il risultato della funzione stessa, lo stampa anche a video.
- 2) Prima di eseguire chiede all'utente se vuole modificare il parametro in ingresso (opzione non possibile se vi sono più parametri)